Beginning Databases with PostgreSQL - Chapter 15: Accessing PostgreSQL from PHP

By: Wrox Press

November 21st, 2001

Reader Rating: 9

About the Author

Wrox Press

View all articles by Wrox Press...

This article is an excerpt of the book, Beginning Databases with PostgreSQL [1] (Wrox Press, 2001), and is reprinted here by permission.

Recently, there has been a strong trend towards providing web-based interfaces to online databases. There are a number of reasons supporting this movement, including:

- Web browsers are common and familiar interfaces for browsing data
- Web-based applications can easily be integrated into an existing web site
- Web (HTML) interfaces are easily created and modified

In this chapter, we will explore various methods for accessing PostgreSQL from PHP. PHP is a server-side, cross-platform scripting language for writing web-based applications. It allows you to embed program logic in HTML pages, which enables you to serve dynamic web pages. PHP allows us to create web-based user interfaces that interact with PostgreSQL.

In this chapter, we will assume at least a basic understanding of the PHP language. If you are completely unfamiliar with PHP, you might want to explore some of the following resources first:

- The home site of PHP http://www.php.net [2]
- *Beginning PHP 4*, Wankyu Choi, Allan Kent, Ganesh Prasad, and Chris Ullman, with Jon Blank and Sean Cazzell, Wrox Press (ISBN 1-861003-73-0)

There are many different schools of thought concerning PHP development methodologies. It is not within the scope of this book to discuss them. Instead, we will focus on designing PHP scripts that make effective use of PHP's PostgreSQL interface.

Note that we will be focusing on PHP version 4. While most of the following code examples and descriptions will apply to earlier versions of PHP, there may be a few differences in functionality. In addition, it is assumed that all code snippets fall within the context of valid PHP scope (generally meaning with the `<?php ?>` tags), unless otherwise specified.

Adding PostgreSQL Support to PHP

Before you can begin developing PHP scripts that interface with a PostgreSQL database, you will need to include PostgreSQL support in your PHP installation.

If you're not sure whether your existing PHP installation already has PostgreSQL support, create a simple script named `phpinfo.php` (which should be placed in your web server's document root), containing the following line:

```
<?php
phpinfo();
?>
```

Examine the output of this script in your web browser. If PostgreSQL support has already been included, the output will contain a section similar to the following:

pgsql

PostgreSQL Support	enabled
Active Persistent Links	0
Active Links	0

Directive	Local Value	Master Value
pgsql.allow_persistent	On	On
pgsql.max_links	Unlimited	Unlimited
pgsql.max_persistent	Unlimited	Unlimited

If your PHP installation already has PostgreSQL support, you can continue on to the next section.

If you have the PHP source code, it is fairly easy to add PostgreSQL support. Simply pass the `--with-pgsql` option to the `configure` script:

```
$ ./configure --with-pgsql
```

You can optionally specify the directory of your PostgreSQL installation if the `configure` script is unable to locate it by itself:

```
$ ./configure --with-pgsql=/var/lib/pgsql
```

Remember that you might need to pass additional options to the `configure` script depending on your build requirements. For example, to build PHP with support for PostgreSQL, LDAP, and XML, you would use the following command line:

```
$ ./configure --with-pgsql --with-imap --enable-xml
```

Refer to the PHP documentation (specifically the `INSTALL` document included with the PHP distribution) for additional compilation options and installation instructions. You can also find them at http://www.php.net/manual/en/html/installation.html [3].

Using the PHP API for PostgreSQL

All of the interaction with the PostgreSQL database is performed through the PostgreSQL extension, which is a comprehensive set of PHP functions. For a complete list of functions and further information about the same, refer to http://www.php.net/manual/ref.pgsql.php [4].

A simple PHP script that opens a connection to a PostgreSQL database, selects some rows, prints the number of rows in the resultset, and closes the connection would look something like this:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = "SELECT * FROM item";
$result = pg_exec($db_handle, $query);
echo "Number of rows: " . pg_numrows($result);
pg_freeresult($result);
pg_close($db_handle);
?>
```

As you can see, interacting with the database from within PHP is fairly straightforward. We will now cover the various aspects of the PHP PostgreSQL extension in more depth.

Database Connections

Before you can interact with the database, you must first open a connection to it. Each connection is represented by a single variable (we'll refer to this variable as the **connection handle**). PHP allows you to have multiple connections open at once, each with its own connection handle.

pg_connect()

Database connections are opened using the `pg_connect()` function. This function takes a connect string as its only argument and returns a database connection handle. Here's an example:

```
$db_handle = pg_connect("dbname=bpsimple user=jon");
```

You can create your own user name and use it to connect to the database as `user=<username>`.

If you want to use PHP variables, remember to surround the connection string in double quotes instead of single quotes:

```
$db_handle = pg_connect("dbname=$dbname user=$dbuser");
```

All of the standard PostgreSQL connection parameters are available in the connection string. The most commonly used options and their meanings are listed below:

- `Dbname`: Database to connect to (Default: `$PGDATABASE`)
- `User`: User name to use when connecting (Default: `$PGUSER`)
- `password`: Password for the specified user (Default: `$PGPASSWORD` or none)
- `Host`: Name of the server to connect to (Default: `$PGHOST` or `localhost`)
- `hostaddr`: IP address of the server to connect to (Default: `$PGHOSTADDR`)
- `Port`: TCP/IP port to connect to on the server (Default: `$PGPORT` or `5432`)

If the connection attempt fails, the `pg_connect()` function will return false. Failed connection attempts can, thus, be detected by testing the return value:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
if ($db_handle) {
   echo 'Connection attempt succeeded.';
} else {
   echo 'Connection attempt failed.';
}
pg_close($db_handle);
?>
```

As mentioned above, PHP supports multiple concurrent database connections:

```
$db_handle1 = pg_connect("dbname=database1");
$db_handle2 = pg_connect("dbname=database2");
```

Persistent Connections

PHP also supports **persistent** database connections. Persistent connections are held open beyond the lifetime of the page request, whereas normal connections are closed at the end of the page request. PHP maintains a list of currently open connections and, if a request is made for a new persistence database connection with the same connection parameters as one of the open connections in this list, a handle to the already opened connection is returned instead. This has the advantage of saving the script the additional overhead of creating a new database connection when a suitable one already exists in the connection pool.

pg_pconnect()

To open a persistent connection to PostgreSQL, use the `pg_pconnect()` function. This function behaves exactly like the `pg_connect()` function described above, except that it requests a persistent connection, if one is available.

It is suggested however that you use persistent connections with care. Overusing persistent connections could lead to a large number of idle database connections to your database. The ideal use of a persistent connection is in those instances where multiple pages will also request the same kind of database connection (meaning one containing same connection parameters). In such cases, persistent connections offer a substantial performance boost.

Closing Connections

pg_close()

Database connections can be explicitly closed using the `pg_close()` function:

```
pg_close($db_handle);
```

There are a few things however that need to be pointed out here. Firstly, in the case of persistent connections, this function will not actually close the connection. Instead, the connection will just be returned to the database connection pool. Secondly, PHP will automatically close any open non-persistent database connections at the end of the script's execution. Both of these points make calling `pg_close()` largely unnecessary, but the function is included for completeness and for those instances where there is truly a need to close the connection immediately.

If the provided connection handle is invalid, `pg_close()` will return false. Otherwise, `pg_close()` will return true upon success.

Connection Information

PHP provides a number of simple functions for retrieving information on the current database connection based on the connection handle provided. Such functions include:

- `pg_dbname()` Returns the name of the current database
- `pg_host()` Returns the hostname associated with the current connection
- `pg_options()` Returns the options associated with the current connection
- `pg_port()` Returns the port number of the current connection
- `pg_tty()` Returns the TTY name associated with the current connection

All of these functions require a connection handle as their sole argument and will return either a string or a number upon success. Otherwise, they will return false:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
echo "<h1>Connection Information</h1>";
echo "Database name: ' . pg_dbname($db_handle) . "<br>\n";
echo "Hostname: " . pg_host($db_handle) . "<br>\n";
echo "Options: " . pg_options($db_handle) . "<br>\n";
echo "Port: " . pg_port($db_handle) . "<br>\n";
echo "TTY name: " . pg_tty($db_handle) . "<br>\n";
pg_close($db_handle);
?>
```

Building Queries

We have already seen a simple example of executing a query from PHP. In this section, we will cover the topic of query building and execution in more depth.

SQL queries are merely strings, so they can be built using any of PHP's string functions. The following are three examples of query string construction in PHP:

```
$lastname = strtolower($lastname);
$query = "SELECT * FROM customer WHERE lname = '$lastname'";
```

This example performs the lower case conversion of `$lastname` first. Then, it builds the query string using PHP's standard string syntax.

Note that the value of `$lastname` will remain lower case after these lines.

```
$query = "SELECT * FROM customer WHERE lname = '" .
        strtolower($lastname) . "'";
```

This example uses an inline call to `strtolower()`. Functions can't be called from inside string literals (in other words between quotation marks), so we need to break our query string into two pieces and concatenate them (using the "dot" operator) with the function call in between.

Unlike the previous example, the result of the `strtolower()` function will not affect the value of `$lastname` after this line is executed by PHP.

```
$query = sprintf("SELECT * FROM customer WHERE lname = '%s'",
                strtolower($lastname));
```

This final example uses the `sprintf()` function to generate the query string. The `sprintf()` function uses special character combinations (the `%s` in the above line, for example) to format strings. More information on the `sprintf()` function is available at http://www.php.net/manual/en/function.sprintf.php [5].

Each of these approaches will produce exactly the same query string. The best method to use, like most things, will depend on the situation. For simple queries, a direct string assignment will probably work best, but when the situation calls for the interpolation or transformation of a large number of variables, you might want to explore different approaches. In some cases, you might encounter a trade-off between execution speed and code readability. This is true of most programming tasks, so you will have to apply your best judgment.

Here's an example of a complex query written as a long assignment string:

```
$query = "UPDATE table $tablename SET " . strtolower($column) . " = '" .
        strtoupper($value) . "'";
```

This could be rewritten using the PHP `sprintf()` function as:

```
$query = sprintf("UPDATE table %s SET %s = '%s'", $tablename,
                strtolower($column), strtoupper($value));
```

The second expression is clearly more readable than the first, although benchmarking will show that this readability comes at a slight performance cost as programmer time is much more expensive than machine time. In this case, the tradeoff of readability over execution speed is probably worth it, unless you are doing hundreds of these types of string constructions per page request.

Complex Queries

In an ideal world, all of our queries would be as simple as those used in the previous examples, but we all know that is seldom true. In those cases where more complex queries need to be built, we find that PHP offers a number of convenient functions to aid us in our task.

For example, consider the case where a large number of table deletions need to be performed. In raw SQL, the query might look something like this:

```
DELETE FROM items WHERE item_id = 4 OR item_id = 6
```

Now, that query alone doesn't appear all that complicated, but what if this query needed to delete a dozen rows, specifying the `item_id` of each row in the `WHERE` clause? The query string gets pretty long at that point, and because the number of expressions in the where clause probably needs to be varying we need to account for these details in our code.

We will probably be receiving our list of item IDs to be deleted from the user via some method of HTML form input, so we can assume they will be stored in some kind of array format (at least, that's the most convenient means of storing the list). We'll assume this array of item IDs is named `$item_ids`. Based on that assumption, the above query could be constructed as follows:

```
<?php
$query = "DELETE FROM items WHERE ";
$query .= "item_id = " . $item_ids[0];
if (count($item_ids) > 1) {
   array_shift($item_ids);
   $query .= " or item_id = " .
             implode(" or item_id =", $item_ids);
}
?>
```

This will produce an SQL query with an arbitrary number of item IDs. Based on this code, we can write a generic function to perform our deletions:

```
<?php

function sqlDelete($tablename, $column, $ids) {
```

```
    $query = '';
    if (is_array($ids)) {
        $query = "DELETE FROM $tablename WHERE ";
        $query .= "$column = " . $ids[0];
        if (count($ids) > 1) {
            array_shift($ids);
            $query .= " or $column = " .
                      implode(" or $column =", $ids);
        }
    }
    return $query;
}

?>
```

Executing Queries

Once the query string has been constructed, the next step is to execute it. Queries are executed using the `pg_exec()` function.

`pg_exec()`

The `pg_exec()` function is responsible for sending the query string to the PostgreSQL server and returning the resultset.

Here's a simple example to illustrate the use of `pg_exec()`:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = 'SELECT * FROM customer'
$result = pg_exec($db_handle, $query);
pg_close($db_handle);
?>
```

As you can see, `pg_exec()` requires two parameters: an active connection handle and a query string. You should already be familiar with each of these from the previous sections. `pg_exec()` will return a resultset upon successful execution of the query. We will work with resultsets in the next section.

If the query should fail, or if the connection handle is invalid, `pg_exec()` will return false. It is therefore prudent to test the return value of `pg_exec()` so that you can detect such failures.

The following example includes some result checking:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = "SELECT * FROM customer";
$result = pg_exec($db_handle, $query);
if ($result) {
    echo "The query executed successfully.<br>\n";
} else {
    echo "The query failed with the following error:<br>\n";
    echo pg_errormessage($db_handle);
}
pg_close($db_handle);
?>
```

In this example, we test the return value of `pg_exec()`. If it is not false (in other words it has a value), `$result` represents a resultset. Otherwise, if `$result` is false, we know that an error has occurred. We can then use the `pg_errormessage()` function to print a descriptive message for that error. We will cover error messages in more detail later in this chapter.

Working with Resultsets

Upon successful execution of a query, pg_exec() will return a resultset identifier, through which we can access the resultset. The resultset stores the result of the query as returned by the database. For example, if a selection query were executed, the resultset would contain the resulting rows.

PHP offers a number of useful functions for working with resultsets. All of them take a resultset identifier as an argument, so they can only be used after a query has been successfully executed. We learned how to test for successful execution in the previous section.

`pg_numrows()` and `pg_numfields()`

Now we'll start with the two simplest result functions: `pg_numrows()` and `pg_numfields()`. These two functions return the number of rows and the number of fields in the resultset, respectively. For example:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = "SELECT * FROM customer";
$result = pg_exec($db_handle, $query);
if ($result) {
   echo "The query executed successfully.<br>\n";
   echo "Number of rows in result: " . pg_numrows($result) . "<br>\n";
   echo "Number of fields in result: " . pg_numfields($result);
} else {
   echo "The query failed with the following error:<br>\n";
   echo pg_errormessage($db_handle);
}
pg_close($db_handle);
?>
```

These functions will return -1 if there is an error.

pg_cmdtuples()

There's also the `pg_cmdtuples()` function, which will return the number of rows **affected** by the query. For example, if we were performing insertions or deletions with our query, we wouldn't actually be retrieving any rows from the database, so the number of rows or fields in the resultset would not be indicative of the query's result. Instead, the changes take place inside of the database. `pg_cmdtuples()` will return the number of rows that were affected by these types of queries (in other words the number of rows inserted, deleted, or updated):

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = "DELETE FROM item WHERE cost_price > 10.00";
$result = pg_exec($db_handle, $query);
if ($result) {
   echo "The query executed successfully.<br>\n";
   echo "Number of rows deleted: " . pg_cmdtuples($result);
} else {
   echo "The query failed with the following error:<br>\n";
   echo pg_errormessage($db_handle);
}
pg_close($db_handle);
?>
```

The `pg_cmdtuples()` function will return 0 if no rows in the database were affected by the query, as in the case of a selection query.

Extracting Values from Resultsets

There are a number of ways to extract values from resultsets. We will start with the `pg_result()` function.

pg_result()

The `pg_result()` function is used when you want to retrieve a single value from a resultset. In addition to a resultset identifier, you must also specify the row and field that you want to retrieve from the result. The row is specified numerically, while the field may be specified either by name or by numeric index. Numbering always starts at zero.

Here's an example using `pg_result()`:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = "SELECT title, fname, lname FROM customer";
$result = pg_exec($db_handle, $query);
if ($result) {
   echo "The query executed successfully.<br>\n";
   for ($row = 0; $row < pg_numrows($result); $row++) {
       $fullname = pg_result($result, $row, 'title') . " ";
       $fullname .= pg_result($result, $row, 'fname') . " ";
       $fullname .= pg_result($result, $row, 'lname');
       echo "Customer: $fullname<br>\n";
   }
} else {
   echo "The query failed with the following error:<br>\n";
   echo pg_errormessage($db_handle);
}
pg_close($db_handle);
?>
```

Using numeric indices, this same block of code could also be written like this:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = "SELECT title, fname, lname FROM customer";
$result = pg_exec($db_handle, $query);
if ($result) {
   echo "The query executed successfully.<br>\n";
   for ($row = 0; $row < pg_numrows($result); $row++) {
       $fullname = "";
       for ($col = 0; $col < pg_numfields($result); $col++) {
           $fullname .= pg_result($result, $row, $col) . " ";
       }
       echo "Customer: $fullname<br>\n";
   }
} else {
   echo "The query failed with the following error:<br>\n";
   echo pg_errormessage($db_handle);
}
pg_close($db_handle);
?>
```

The first example is a bit more readable, however, and doesn't depend on the order of the fields in the resultset. PHP also offers more advanced ways of retrieving values from resultsets, because iterating through rows of results isn't especially efficient.

pg_fetch_row()

PHP provides two functions, `pg_fetch_row()` and `pg_fetch_array()`, that can return multiple result values at once. Each of these functions returns an array.

`pg_fetch_row()` returns an array that corresponds to a single row in the resultset. The array is indexed numerically, starting from zero. Here is the previous example rewritten to use `pg_fetch_row()`:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = "SELECT title, fname, lname FROM customer";
$result = pg_exec($db_handle, $query);
if ($result) {
   echo "The query executed successfully.<br>\n";
   for ($row = 0; $row < pg_numrows($result); $row++) {
       $values = pg_fetch_row($result, $row);
       $fullname = "";
       for ($col = 0; $col < count($values); $col++) {
           $fullname .= $values[$col] . " ";
       }
       echo "Customer: $fullname<br>\n";
   }
} else {
   echo "The query failed with the following error:<br>\n";
   echo pg_errormessage($db_handle);
}
pg_close($db_handle);
?>
```

As you can see, using `pg_fetch_row()` eliminates the multiple calls to `pg_result()`. It also places the result values in an array, which can be easily manipulated using PHP's native array functions.

In this example, However, we are still accessing the fields by their numeric indices. Ideally, we should also be able to access each field by its associated name. To accomplish that, we can use the `pg_fetch_array()` function.

pg_fetch_array()

The `pg_fetch_array()` function also returns an array, but it allows us to specify whether we want that array indexed numerically or associatively (using the field names as keys). This preference is specified by passing one of the following as the third argument to `pg_fetch_array()`:

- `PGSQL_ASSOC` Index the resulting array by field name
- `PGSQL_NUM` Index the resulting array numerically
- `PGSQL_BOTH` Index the resulting array both numerically and by field name

If you don't specify one of the above indexing methods, `PGSQL_BOTH` will be used by default. Note that this will double the size of your resultset, so

you're probably better off explicitly specifying one of the above. Also note, that the field names will always be returned in lower case letters, regardless of how they're represented in the database itself.

Here's the example rewritten once more, now using `pg_fetch_array()`:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = "SELECT title, fname, lname FROM customer";
$result = pg_exec($db_handle, $query);
if ($result) {
   echo "The query executed successfully.<br>\n";
   for ($row = 0; $row < pg_numrows($result); $row++) {
       $values = pg_fetch_array($result, $row, PGSQL_ASSOC);
       $fullname = $values['title'] . " ";
       $fullname .= $values['fname'] . " ";
       $fullname .= $values['lname'];
       echo "Customer: $fullname<br>\n";
   }
} else {
   echo "The query failed with the following error:<br>\n";
   echo pg_errormessage($db_handle);
}
pg_close($db_handle);
?>
```

pg_fetch_object()

PHP also allows you to fetch the result values with the `pg_fetch_object()` function. Each field name will be represented as a property of this object. Thus, fields cannot be accessed numerically. Written using `pg_fetch_object()`, our example looks like this:

```
<?php
$db_handle = pg_connect("dbname=bpsimple");
$query = "SELECT title, fname, lname FROM customer";
$result = pg_exec($db_handle, $query);
if ($result) {
   echo "The query executed successfully.<br>\n";
   for ($row = 0; $row < pg_numrows($result); $row++) {
       $values = pg_fetch_object($result, $row, PGSQL_ASSOC);
       $fullname = $values->title . " ";
       $fullname .= $values->fname . " ";
       $fullname .= $values->lname;
       echo "Customer: $fullname<br>\n";
   }
} else {
   echo "The query failed with the following error:<br>\n";
   echo pg_errormessage($db_handle);
}
pg_close($db_handle);
?>
```

Field Information

PHP allows you to gather some information on the field values in your resultset. These functions may be useful in certain circumstances, so we will cover them briefly here.

pg_fieldisnull()

PostgreSQL supports a notion of `NULL` field values. PHP doesn't necessarily define `NULL` the same way PostgreSQL does, however. To account for this, PHP provides the `pg_fieldisnull()` function so that you may determine whether a field value is `NULL` based on the PostgreSQL definition of `NULL`:

```
if (pg_fieldisnull($result, $row, $field)) {
   echo "$field is NULL.";
} else {
   echo "$field is " . pg_result($result, $row, $field);
}
```

pg_fieldname() and pg_fieldnum()

These functions return the name or number of a given field. The fields are indexed numerically, starting with zero:

```
echo "Field 1 is named: " . pg_fieldname($result, 1);
echo "Field item_id is number: " . pg_fieldnum($result, "item_id");
```

Note that `pg_fieldname()` will return the field name as specified in the `SELECT` statement.

`pg_fieldsize()`, `pg_fieldprtlen()`, and `pg_fieldtype()`

The size, printed (character) length, and type of fields can be determined:

```
echo "Size of field 2:" . pg_fieldsize($result, 2);
echo "Length of field 2: " . pg_fieldprtlen($result, $row, 2);
echo "Type of field 2: " . pg_fieldtype($result, 2);
```

As usual, the numeric field indices start at zero. Field indices may also be specified as a string representing the field name.

Also, if the size of the field is variable, `pg_fieldsize()` will return a -1, or false on an error. `pg_fieldprtlen()` will return -1 on an error.

Freeing Resultsets

`pg_freeresult()`

It is possible to free the memory used by a resultset by using the `pg_freeresult()` function:

```
pg_freeresult($result);
```

PHP will automatically free up all result memory at the end of the script's execution anyway, so this function only needs to be called if you're especially worried about memory consumption in your script, and you know you won't be using this resultset again later on in your script's execution.

Type Conversion of Result Values

PHP does not offer the diverse data type support you might find in other languages, so values in resultsets are sometimes converted from their original data type to a PHP-native data type. For the most part, this conversion will have very little or no effect on your application, but it's important to be aware that some type conversion may occur:

- All integer, boolean, and OID types are converted to integers
- All forms of floating point numbers are converted to doubles
- All other types (arrays, etc.) are represented as strings

Error Handling

We touched on error handling very briefly in an earlier section. We will now cover it in a bit more detail.

Nearly all PostgreSQL related functions return some sort of predictable value upon an error (generally false or -1). This makes it fairly easy to detect error situations so that your script can fail gracefully. For example:

```
$db_handle = pg_connect('dbname=bpsimple');
if (!$db_handle) {
   header("Location: http://www.example.com/error.php");
   exit;
}
```

In the above example, the user was to be redirected to an error page if the database connection attempt were to fail.

`pg_errormessage()`

`pg_errormessage()` can be used to retrieve the text of the actual error message as returned by the database server. `pg_errormessage()` will always return the text of the last error message generated by the server. Be sure to take that into consideration when designing your error handling and display logic.

You will find that, depending on your level of error reporting, PHP can be fairly verbose when an error occurs, often outputting several lines of errors and warnings. In a production environment, it is often undesirable to display this type of message to the end user.

The @ Symbol

The most direct solution is to lower the level of error reporting in PHP (controlled via the `error_reporting` configuration variable in the `php.ini`). The second option is to suppress these error messages directly from PHP code on a per-function-call basis. PHP uses the @ symbol to indicate error suppression. For example, no errors will be output from the following code:

```
$db_handle = pg_connect("host=nonexistent_host");
$result = @pg_exec($db_handle, "SELECT * FROM item");
```

Without the @ symbol, the second line above would generate an error complaining about the lack of a valid database connection (assuming your error reporting level was high enough to cause that error to be displayed, of course).

Note that the above error could still be detected by testing the value of `$result`, though, so suppressing the error message output doesn't preclude our dealing with error situations programmatically. Furthermore, we could display the error message at our convenience using the `pg_errormessage()` function.

Character Encoding

If character encoding support is enabled in PostgreSQL, PHP provides functions for getting and setting the current client encoding. By default, the encoding is set to `SQL ASCII`.

The supported character sets are: `SQL_ASCII`, `EUC_JP`, `EUC_CN`, `EUC_KR`, `EUC_TW`, `UNICODE`, `MULE_INTERNAL`, `LATINX (X=1...9)`, `KOI8`, `WIN`, `ALT`, `SJIS`, `BIG5`, `WIN1250`.

`pg_client_encoding()`

The `pg_client_encoding()` function will return the current client encoding:

```
$encoding = pg_client_encoding($db_handle);
```

`pg_set_client_encoding()`

You can set the current client encoding using the `pg_set_client_encoding()` function:

```
pg_set_client_encoding($db_handle, 'UNICODE');
```

PEAR

PEAR (The PHP Extension and Application Repository) is an attempt to replicate the functionality of Perl's CPAN in the PHP community. To quote the official PEAR goals:

- To provide a consistent means for library code authors to share their code with other developers
- To give the PHP community an infrastructure for sharing code
- To define standards that help developers write portable and reusable code
- To provide tools for code maintenance and distribution

PEAR is primarily a large collection of PHP classes, which make use of PHP's object-oriented programming capabilities. You will therefore, need to become familiar with PHP's syntax for working with classes. PHP's object-oriented extensions are documented here: http://www.php.net/manual/en/language.oop.php [6].

More information on PEAR is available at:

- http://pear.php.net [7]
- http://php.weblogs.com/php_pear_tutorials/ [8]

PEAR's Database Abstraction Interface

PEAR includes a database (DB) abstraction interface, which is included with the standard PHP distribution. The advantage to using a database abstraction interface instead of calling the database's native functions directly is code independence. Should you need to move your project to a different database, it would probably involve a major code rewrite. If you had used a database abstraction interface, however, the task would be trivial.

PEAR's DB interface also adds some value-added features, such as convenient access to multiple resultsets and integrated error handling. All of the database interaction is handled through the DB classes and objects. This is conceptually similar to Perl's DBI interface.

The main disadvantage to a database abstraction interface is the performance overhead it incurs on your applications execution. Once again, this is a situation where there is a trade-off between code flexibility and performance.

Using the DB Interface

The following example illustrates the use of the DB interface: note that this example assumes that the PEAR DB interface has already been installed and that it can be found via the current `include_path` setting. Both of these are the default for newer PHP4 installations:

```
<?php

/* Import the PEAR DB interface. */
require_once "DB.php";

/* Database connection parameters. */
$username = "jon";
```

```
$password = "secret";
$hostname = "localhost";
$dbname = "bpsimple";

/* Construct the DSN -- Data Source Name. */
$dsn = "pgsql://$username:$password@$hostname/$dbname";

/* Attempt to connect to the database. */
$db = DB::connect($dsn);

/* Check for any connection errors. */
if (DB::isError($db)) {
   die ($db->getMessage());
}

/* Execute a selection query. */
$query = "SELECT title, fname, lname FROM customer";
$result = $db->query($query);

/* Check for any query execution errors. */
if (DB::isError($result)) {
   die ($result->getMessage());
}

/* Fetch and display the query results. */
while ($row = $result->fetchRow(DB_FETCHMODE_ASSOC)) {
   $fullname = $row['title'] . " ";
   $fullname .= $row['fname'] . " ";
   $fullname .= $row['lname'];
   echo "Customer: $fullname<br>\n";
}

/* Disconnect from the database. */
$db->disconnect();

?>
```

As you can see, this code, while not using any PostgreSQL functions directly, still follows the same programmatic logic of our previous examples. It is also easy to see how the above example could easily be adapted to use another type of database (Oracle or MySQL, for example) without much effort.

PEAR's Error Handling

Using the PEAR DB interface offers you as a developer a number of additional advantages. For example, PEAR includes an integrated error handling system. Here is some code to demonstrate error handling:

```
<?php

/* Import the PEAR DB interface. */
require_once 'DB.php';

/* Construct the DSN -- Data Source Name. */
$dsn = "pgsql://jon:secret@localhost/bpsimple";

/* Attempt to connect to the database. */
$db = DB::connect($dsn);

/* Check for any connection errors. */
if (DB::isError($db)) {
   die ($db->getMessage());
}
```

Above, we see the first instance of PEAR's error handling capabilities: `DB::isError()`. If the call to `DB::connect()` fails for some reason, it will return an `PEAR_Error` instance, instead of a database connection object. We can test for this case using the `DB::isError()` function, as shown above.

Knowing an error occurred is important, but finding out why that error occurred is even more important. We can retrieve the text of the error message (in this case, the connection error generated by PostgreSQL) using the `getMessage()` method of the `PEAR_Error` object. This is also demonstrated in the example above.

Our example continues with some queries:

```
/* Make errors fatal. */
$db->setErrorHandling(PEAR_ERROR_DIE);

/* Build and execute the query. */
$query = "SELECT title, fname, lname FROM customer";
$result = $db->query($query);

/* Check for any query execution errors. */
if (DB::isError($result)) {
   die ($result->getMessage());
}

while ($row = $result->fetchRow(DB_FETCHMODE_ASSOC)) {
   $fullname = $row['title'] . " ";
   $fullname .= $row['fname'] . " ";
   $fullname .= $row['lname'];
   echo "Customer: $fullname<br>\n";
}

/* Disconnect from the database. */
$db->disconnect();

?>
```

Note that we have changed PEAR's error handling behavior with the call to the `setErrorHandling()` method. Setting the error handling behavior to `PEAR_ERROR_DIE` will cause PHP to exit fatally if an error occurs.

Here's a list of the other error handling behaviors:

- `PEAR_ERROR_RETURN` Simply return an error object (default)
- `PEAR_ERROR_PRINT` Print the error message and continue execution
- `PEAR_ERROR_TRIGGER` Use PHP's `trigger_error()` function to raise an internal error
- `PEAR_ERROR_DIE` Print the error message and abort execution
- `PEAR_ERROR_CALLBACK` Use a callback function to handle the error before aborting execution

Additional information on the `PEAR_Error` class and PEAR error handling is available here: http://php.net/manual/en/class.pear-error.php [9].

Query Preparation and Execution

PEAR also includes a handle method of preparing and executing queries. Here's an abbreviated example demonstrating the `prepare()` and `execute()` methods of the DB interface. This example assumes we already have a valid database connection (from a `DB::connect()`):

```
/* Set up the $items array. */
$items = array(
   '6241527836190' => 20,
   '7241427238373' => 21,
   '7093454306788' => 22
);

/* Prepare our template SQL statement. */
$statement = $db->prepare("INSERT INTO barcode VALUES(?,?)");

/* Execute the statement for each entry in the $items array. */
while (list($barcode, $item_id) = each($items)) {
   $db->execute($statement, array($barcode, $item_id));
}
```

This example probably requires some explanation for those of you who are unfamiliar with prepared SQL statements.

The call to the `prepare()` method creates a SQL template that can be executed repetitively. Note the two wildcard spots in the statement that are specified using question marks. These placeholders will be replaced with actual values later on when we call the `execute()` method.

Assuming we have an array of `$items` that contain barcodes and item IDs, we will want to perform one database insertion per item. To accomplish this, we construct a loop to iterate over each entry in the `$items` array, extract the barcode and item ID, and then execute the prepared SQL statement.

As mentioned above, the `execute()` method will replace the placeholder values in the prepared statement with those values passed to it in the second argument in array form. In the above example, this would be the `array($barcode, $item_id)` argument. The placeholder values are replaced in the order these new values are specified, so it's important to get them right.

Hopefully, you'll find this feature of the PEAR DB interface very useful in your own projects.

In this chapter, we examined the various ways that a PostgreSQL database can be accessed from the PHP scripting language.

We covered the various aspects of database connections, query building and execution, resultset manipulation, and error handling. We also introduced the PEAR database abstraction interface.

From this foundation, you should now have enough of the basic tools to begin developing your own web-based database applications.

[1] http://www.wrox.com/Books/Book_Details.asp?section=1_1&isbn=1861005156&subject=&subject_id=
[2] http://www.php.net
[3] http://www.php.net/manual/en/html/installation.html
[4] http://www.php.net/manual/ref.pgsql.php
[5] http://www.php.net/manual/en/function.sprintf.php
[6] http://www.php.net/manual/en/language.oop.php
[7] http://pear.php.net
[8] http://php.weblogs.com/php_pear_tutorials/
[9] http://php.net/manual/en/class.pear-error.php